

Proyecto 1

Estudiante: Fabricio González Cerdas, Jian Zheng Wu, Jafet Diaz Morales

Curso: Arquitectura de Computadores I

Profesor: Jeferson González Cerdas

Contents

1	Especificación de la Arquitectura	2
1.1	Tamaño de palabra e instrucción	2
1.2	Conjunto de registros con sus nombres y propósitos	2
1.3	Organización de memoria	3
1.4	Tabla completa de instrucciones con opcodes, formatos y ejemplos	3

1 Especificación de la Arquitectura

1.1 Tamaño de palabra e instrucción

Todas las instrucciones de la arquitectura Craft21 tienen una longitud fija de 32 bits. Esto permite que el proceso de decodificación sea más simple, ya que cada instrucción ocupa exactamente una palabra de instrucción.

Cuando un formato de instrucción no utiliza los 32 bits completos, los bits restantes se rellenan con ceros a la izquierda. Los registros generales y los registros Vault se codifican con campos de 5 bits, lo que permite representar hasta 32 registros por banco.

La arquitectura utiliza principalmente los siguientes campos:

- **opcode**: identifica el tipo general de instrucción.
- **func**: especifica la operación concreta dentro de un mismo formato.
- **rd**: registro destino.
- **rs1**: primer registro fuente.
- **rs2**: segundo registro fuente.
- **imm**: valor inmediato utilizado en instrucciones de carga, almacenamiento, saltos o constantes.

1.2 Conjunto de registros con sus nombres y propósitos

Table 1: Registros Craft21

Registros Craft21				
Índice	Register	ABI Name	Description	Saver
1	x0	zero	Wired zero	-
2	x1	ra	Return address	Caller
3	x2	sp	Stack pointer	Callee
4-11	x3-10	t0-t7	Temporaries	Caller
12-13	x11-12	a0-a1	Return Values / function arguments	Caller
14-17	x13-16	a2-a5	Extra function arguments	Caller
18	x17	s0/fp	Frame pointer / saved register	Callee
19-20	x18-19	reserved	Reserved registers	-
21-32	x20-31	s1-s12	Saved registers	Callee
1'	v0	rpass	Password container	VaultCaller
2'-5'	v1-4	key0-3	Key registers	VaultCaller
6'-32'	v5-31	reserved	Extra Vault registers	VaultCaller

La arquitectura Craft21 cuenta con dos grupos principales de registros: registros generales y registros Vault. Los registros generales se utilizan para operaciones aritméticas, lógicas, manejo de funciones, parámetros y variables temporales. Los registros Vault están pensados

para operaciones especiales relacionadas con almacenamiento protegido, manejo de claves y operaciones internas de seguridad.

El registro **x0** funciona como un registro constante con valor cero. El registro **x1** almacena la dirección de retorno en llamadas a función. El registro **x2** funciona como puntero de pila. Los registros temporales **t0-t7** pueden ser modificados por el llamador, mientras que los registros guardados **s0-s12** deben conservarse entre llamadas a función.

1.3 Organización de memoria

La arquitectura Craft21 organiza la memoria en espacios separados para instrucciones, datos, pila y bóveda de llaves. El segmento de instrucciones se almacena en la ROM de instrucciones, mientras que los datos normales se almacenan en la RAM de datos. Además, existe una memoria separada para la bóveda, utilizada por las instrucciones especiales Vault.

El segmento **.text** contiene las instrucciones del programa. Cada instrucción ocupa 32 bits, es decir, 4 bytes. Por esta razón, el contador de programa avanza de cuatro en cuatro. La ROM de instrucciones se accede por palabra, usando la dirección del PC dividida entre cuatro para seleccionar la instrucción correspondiente.

El segmento **.data** contiene variables globales e inicializadores estáticos. En la versión actual del compilador, las variables globales se asignan desde la dirección base **0x8000**. Los valores se emiten usando directivas como **.word**, **.half**, **.byte** y **.space**. La memoria de datos se accede por byte, pero las instrucciones **lw** y **sw** permiten leer y escribir palabras completas de 32 bits.

La pila utiliza los registros **x2** y **x17**. El registro **x2** funciona como *stack pointer* y se inicializa en la dirección **0x7FF0**. El registro **x17** funciona como *frame pointer*. Al entrar a una función, el generador de código reserva espacio para el marco de pila, guarda la dirección de retorno **ra** en **0(sp)** y guarda el **fp** anterior en **4(sp)**. Luego calcula el nuevo **fp** como **sp + frame_size**.

Los parámetros de función se reciben inicialmente en registros de argumento y luego se guardan en el marco de pila con offsets negativos respecto a **x17**. Las variables locales también se almacenan con offsets negativos desde **x17**. En el caso de parámetros tipo **chest**, no se copia todo el arreglo; se guarda una referencia a la dirección base del arreglo original.

Los arreglos del lenguaje, representados mediante **chest**, se almacenan como bloques contiguos. Para acceder a un elemento, el compilador calcula la dirección base del arreglo y suma el índice multiplicado por el tamaño del tipo. Para tipos de 32 bits, como **int** y **uint32**, el índice se multiplica por cuatro.

Las variables de tipo **chest[ender, N]** se asignan al segmento **VAULT**. Este segmento no utiliza la RAM normal, sino la memoria de bóveda. Su direccionamiento inicia desde offset **0x0000** dentro de dicha memoria protegida. Estas variables se acceden mediante instrucciones especiales como **lwv**, **swv**, **portalv** y **closev**.

1.4 Tabla completa de instrucciones con opcodes, formatos y ejemplos

La arquitectura Craft21 define instrucciones de tipo R, I, S, B y formatos especiales Vault. Las instrucciones tipo R se utilizan principalmente para operaciones entre registros. Las tipo I permiten operaciones con inmediatos. Las tipo S se emplean para carga y almacenamiento en memoria. Las tipo B permiten saltos condicionales. Las instrucciones Vault permiten operaciones especiales sobre registros y memoria protegida.

Las tablas siguientes describen los formatos, opcodes y nemónicos definidos para la ISA Craft21.

Table 2: Operaciones tipo R

Operaciones tipo R					
Bits					Nemónico
23:19	18:14	13:9	8:4	3:0	
func				opcode	
00000	rs2	rs1	rd	0000	add
00001				0000	sub
00010				0000	sll
00011				0000	slt
00100				0000	xor
00101				0000	srl
00110				0000	sra
00111				0000	or
01000				0000	and
01001				0000	mul
01010				0000	div
01011	x0	x0	x0	0000	SLEEP
01100				0000	FREEZE

Table 3: Operaciones tipo S

Operaciones tipo S						
Bits						Nemónico
27:20	func [19]	18:14	13:9	8:4	3:0	
imm[12:5]	0	rs2	rs1	imm[4:0]	opcode	
					0010	sw
	1	imm[4:0]	rs1	rd	0010	lw
	1				0011	lb
	0	rs2	rs1	imm[4:0]	0011	sb

Table 4: Operaciones tipo B

Operaciones tipo B						
Bits						Nemónico
31:24	23:19	18:14	13:9	8:4	3:0	
imm[12:5]	func	rs2	rs1	imm[4:0]	opcode	
	00000				0110	beq
	00001				0110	bne
	00010				0110	blt
	00011				0110	bge

Table 5: Operaciones tipo I

Operaciones tipo I					
Bits					Nemónico
31:16	15:14	13:9	8:4	3:0	
imm [15:0]	func[1:0]	rs1	rd	opcode	
	00			0001	addi
	01			0001	addiHIGH
imm [15:0]	10	rs1	rd	0001	addiSigned
	11			0001	

Table 6: Operaciones PrepareVault (Vault S)

Operaciones PrepareVault (Vault S)						
Bits						Nemónico
27:20	func [19]	18:14	13:9	8:4	3:0	
imm[12:5]	0	rv	rs1	imm[4:0]	opcode	
					1110	swv
	1	imm[4:0]	rs1	rdv	1110	lwv

Table 7: Operaciones In-Vault (Vault Reg)

Operaciones In-Vault (Vault Reg)					
Bits					Nemónico
23:19	18:14	13:9	8:4	3:0	
func				opcode	
00010	rv	rs1	rd	1010	sllv
00101				1010	slrv
00000	rs2	rs1	rdv	1011	changev
00000	zero	x21	zero	1100	closev

Table 8: Operaciones Login (Vault B)

Operaciones Login (Vault B)						
Bits						Nemónico
31:24	23:19	18:14	13:9	8:4	3:0	
imm[12:5]	00100	rv	rs1	imm[3:0]	0110	portalv

Table 9: Operaciones tipo IV

Operaciones tipo IV					
Bits					Nemónico
31:16	15:14	13:9	8:4	3:0	
	func[1:0]			opcode	
imm [15:0]	00	rs1	rdv	1111	addiLOWv
	01			1111	addiHIGHv